UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE MATEMÁTICA E ESTATÍSTICA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Rafael Vieira de Carvalho

Divisão de nós em uma max-tree

Atualização dinâmica de max-trees sob transformações anti-extensivas locais

Orientador: Prof. Dr. Ronaldo F. Hashimoto
Coorientador: Prof. Dr. Wonder A. L. Alves

São Paulo
2026

1 Resumo

2 Introdução e Justificativas

A morfologia matemática é uma subárea do processamento de imagens e da visão computacional que tem como objetivo estudar e desenvolver transformações aplicadas a imagens, tendo como principal arcabouço teórico a teoria dos conjuntos [8]. Conforme sugere o próprio termo *morfologia*, essa área busca analisar e extrair informações relacionadas às características geométricas dos objetos presentes nas imagens, como forma, tamanho e estrutura.

Nessa busca pelo desenvolvimento de transformações, o uso de estrutura de dados, para representar e aplicar transformações em imagens, é extremamente necessário. Exemplos de estruturas são: quad-trees [15, 3], binary partition tree [11], watershed tree [7]

A *component tree* [12] é uma das estruturas utilizadas para representar imagens. A ideia fundamental por trás dessa estrutura consiste em representar uma imagem a partir da ordenação de seu conjunto de pixels e de suas componentes conexas. Para isso, é necessário definir o conceito de componente conexa em uma imagem [10, 9] e, de forma mais geral, estabelecer como uma imagem pode ser representada por meio de um grafo [8].

A representação de uma imagem como um grafo é realizada de forma direta, na qual os vértices correspondem aos pixels da imagem, enquanto as arestas são definidas de acordo com um determinado modelo de conectividade [10].

Conforme dito anteriormente, uma das premissas para a contrução da component tree é a ordenação do conjunto de pixels. Existem duas formas de ordenar os pixels: de maneira ascendente ou de maneira descendente. Essas duas formas dão origem a duas estrutura de dados duais, a max tree cuja ordenação é ascendente, ou seja, a relação de ordenação dos pixels se dá pelo operador $\leq$ entre os níves de cinza. E a min tree cuja ordenação é descendente, ou seja, a relação de ordenaçõs dos pixels se dá pelo operador $\geq$ entre os níveis de cinza.

A component tree originalmente foi desenhada para realizar operações conexas com o propósito de resolver tarefas de filtragem e segmentação [13, 14]. Para realizar tais tarefas a operação mais comum utilizada é o corte, que pode consistir na remoção de alguns nós baseado na otimização de algum critério ou na remoção de alguns nós baseado nos atributos contidos nesses nós.

Tal estrutura é amplamente utilizada devido ao seu baixo custo computacional de construção. Segundo [6], uma imagem com n pixels pode ter sua *component tree* construída em tempo $O(n \log n)$. Além dessa característica, a *component tree* permite extrair informações morfológicas relevantes das imagens. Por esse motivo, essa estrutura tem sido empregada em abordagens de aprendizado profundo para a extração de características, conforme apresentado em [1, 2, 5].

Nesse contexto, uma dificuldade encontrada na utilização de *component trees* é a atualização dessa estrutura quando a imagem sofre alguma alteração. Isto é, quando a imagem sofre alguma alteração não existe nenhum algoritmo para atualizar *component tree* de modo a representar a imagem transformada. Dessa forma, nesse caso de uso se faz necessário reconstruir a árvore novamente para cada mudança que ocorre na imagem.

Em [4] os autore propõem a operação de *merge* (mescla) de nós para resolver parcialmente o problema de atualização. Como apontado também em [4] o problema dual, *split*/divisão, também precisa ser resolvido para poder-se atualizar completamente a árvore.

Sob esse contexto, esse projeto de pesquisa pretende contribuir com essa lacuna na literatura. Para simplificar, assim como [4] trata apenas de *max tree*, esse projeto tratará também apenas desse tipo de estrutura.

Este documento está organizado da seguinte forma. Na seção 3 serão introduzidas algumas das notações, propriedades e definições apresentadas em [4]. Em posse de tal conhecimento teórico, na Seção 4 será apresentada a pergunta de pesquisa de maneira formal, bem como as perguntas auxiliares que a orientam. Na Seção 5, serão descritos os métodos que serão empregados para alcançar os objetivos propostos, bem como os procedimentos adotados para garantir a reprodutibilidade da pesquisa. Na Seção 6, será apresentado o planejamento das atividades e a priorização das etapas a serem executadas, de modo a assegurar o desenvolvimento adequado da pesquisa. Por fim, na Seção 7, serão discutidos os possíveis riscos associados à pesquisa e propostas estratégias de mitigação e alternativas para os casos em que o planejamento originalmente estabelecido não produza os resultados esperados.

3 Notações e teoria

3.1 Conjuntos

Seja A um conjunto. O conjunto potência de A, ou conjunto das partes, é denotado por 2^A. Seja S um conjunto de subconjuntos de 2^A ($S \subseteq 2^A$), escrevemos:

$$\bigcup S = \bigcup_{X \in S} X$$

3.2 Relações

Seja $a,\ b \in A$ e seja $\ltimes$ uma relação binária em A, ou seja, $\ltimes \subseteq A \times A$. Usamos a notação $a \ltimes b$ para expressar o elemento $(a, b) \in \ltimes$.

Denotamos por $\rtimes$ a relação inversa associada a $\ltimes$, isto é, $(a \rtimes b) \iff (b \ltimes a)$

Denotamos que o conjunto de elemento tais que a relação $a \ltimes b$ é verdadeira por:

$$\ltimes(a) = \{b \in A : a \ltimes b\}$$

3.3 Funções

Seja B um subconjunto, uma função (ou uma aplicação) $\mathcal{F}$ de A para B é uma parte de $A \times B$ tal que para qualquer $a \in A$ existe exatamente um $b \in B$ tal que $(a, b) \in \mathcal{F}$. Escrevemos $b = \mathcal{F}(a)$ para expressar tal par ordenado. Denotamos por B^A o conjunto de todas as funções que levam A para B.

Seja $\widetilde{A} \subseteq A$, defini-se o seguinte conjunto $\mathcal{F}(\widetilde{A}) = \{\mathcal{F}(a) : a \in \widetilde{A}\}$. Quando $\mathcal{F}(\widetilde{A}) = \{b\}$ escrevemos $\mathcal{F}(\widetilde{A}) = b$ por abuso de notação. Seja $\widetilde{B} \subseteq B$, escrevemos $\mathcal{F}^{-1}(\widetilde{B}) = \{a \in A : \mathcal{F}(a) \in \widetilde{B}\}$. Quando $\widetilde{B} = \{b\}$ escrevemos $\mathcal{F}^{-1}(b)$ ao invés de $\mathcal{F}^{-1}(\{b\})$ por abuso de notação

3.4 Ordem

Seja $\preccurlyeq$ uma relação binária. Dizemos que $\preccurlyeq$ é uma ordem em A, mais precisamente que $(A, \preccurlyeq)$ é um conjunto ordenado, se a relação é reflexiva, transitiva e antissimétrica. Dizemos que $\preccurlyeq$ é totalmente ordenado se além de ser ordenado também possuí a condição de comparabilidade (ou totalidade). Dizemos que $\preccurlyeq$ é uma ordem parcial em A, ou *poset*, se não possui a condição de comparabilidade.

Suponha que $(A, \preccurlyeq)$ é um conjunto ordenado. Seja $\widetilde{A} \subseteq A$. Define-se o seguinte:

- O supremo de $(\widetilde{A}, \preccurlyeq)$ é denotado por $\bigvee^{\preccurlyeq} \widetilde{A}$
- O ínfimo de $(\widetilde{A}, \preccurlyeq)$ é denotado por $\bigwedge^{\preccurlyeq} \widetilde{A}$
- O conjunto dos elementos maximais de $(A, \preccurlyeq)$ é denotado por $\nabla^{\preccurlyeq} \tilde{A}$
- O conjunto dos elementos minimais de $(A, \preccurlyeq)$ é denotado por $\Delta^{\preccurlyeq} \tilde{A}$

O diagrama de Hasse do conjunto ordenado $(A, \preccurlyeq)$ é denotado por $(A, \lhd)$ e é uma remoção de redundâncidas das propriedades reflexias e transitivas. Por abuso de notação, o diagrama de Hasse no conjunto ordenado $(\tilde{A}, \preccurlyeq)$ também é denotado por $(\tilde{A}, \lhd)$.

Quando $(A, \preccurlyeq)$ é um conjunto totalmente ordenado e $a \preccurlyeq b$ com $a,\ b \in A$, denotamos o intervalo do elementos entre a e b por

$$[\![a, b]\!] = \{c \in A : a \preccurlyeq c \preccurlyeq b\}$$

3.5 Árvores

Seja A um conjunto não vazio e $\preccurlyeq$ uma ordem em A. O conjunto ordenado $(A, \preccurlyeq)$ é considerado de ordem hierárquica superior se as seguintes condições são verdadeiras:

1. $(A, \preccurlyeq)$ admite máximo $\bigvee^{\preccurlyeq} \widetilde{A}$
2. $\forall a \in A$, o conjunto $(\preccurlyeq (a), \preccurlyeq)$ é totalmente ordenado

Sob essas condições, podemos dizer que se $(A, \preccurlyeq)$ é uma ordem hierárquica superior, o diagrama de Hasse $(A, \lhd)$ de tal conjunto ordenado é uma árvore em que:

1. A raíz é $\bigvee^{\preccurlyeq} A$
2. O conjunto de folhas é $\Delta^{\preccurlyeq} A$
3. O conjunto de arestas é $\lhd$

A relação $\lhd$ pode ser vista como uma função $\lhd : A \to A$ e, portanto, podemos escrever como $\lhd(a) = b$ ao invés de $a \lhd b$.

Seja $\emptyset \subset \widetilde{A} \subseteq A$. Uma subárvore de $(A, \lhd)$ é definida como:

$$\overset{\triangle}{A} = \bigcup_{a \in \widetilde{A}} [\![a, \overset{\preccurlyeq}{\bigvee} \widetilde{A}]\!]$$

Ou seja, a árvore $(\overset{\triangle}{A}, \lhd)$ é uma subárvore de $(A, \lhd)$. Uma propriedade importante é, se $\overset{\triangle}{A} \neq A$ então:

- $(A \setminus \overset{\triangle}{A}, \preccurlyeq)$ continua um conjunto ordenado superior
- $(A \setminus \overset{\triangle}{A}, \lhd)$ continua sendo uma árvore

3.6 Árvore de componentes

Com as definições de conjuntos, relações, funções, ordem e árvores agora podemos definir o que é uma imagem e o que é a árvore de componentes dessa imagem.

Definição 1 (relação de adjacência)**.** Seja $\mathbb{P}$ um conjunto finito. Uma relação binária $\frown \subseteq \mathbb{P} \times \mathbb{P}$ é dita de adjacência se for irreflexiva e simétrica.

Definição 2 (relação de conectividade)**.** Uma relação de adjacência $\frown$ induz uma relação de conectividade $\sim$ em $\mathbb{P}$. Dois elementos $x,\ y \in \mathbb{P}$ são ditos conexos ou conectados($x \sim y$) se e somente se $x \frown y$ ou se existe uma sequência finita com $k \geq 1$ formada por elementos $x_0,\ x_1,\ \ldots, x_k \in \mathbb{P}$ tal que:

1. $x_0 = x$
2. $x_k = y$
3. $x_i \frown x_{i+1}$ para todo $i \in \{0,\ 1,\ \ldots, k-1\}$

Definição 3. Um conjunto $X \subseteq \mathbb{P}$ é dito ser uma componente conexa se for maximal a propriedade de conexidade.

Para qualquer $X \subseteq \mathbb{P}$, o conjunto de componentes conexas de X é denotado por $\Pi(X)$ (com $\Pi(\emptyset) = \emptyset$). Note que quando $X \neq \emptyset$, o conjunto $\Pi(X)$ é uma partição de X. Quando $X \neq \emptyset$, usamos $\mathscr{C}(X)$ para expressar que X é um conjunto conexo, ou seja, que possui apenas uma componente conexa ($\Pi(X) = \{X\}$)

Definição 4. Seja $\mathbb{V} \subseteq \mathbb{Z}$ um conjunto finito com a relação de ordem $\leq$. Seja $\perp = \bigwedge^{\leq} \mathbb{V}$ e $\top = \bigvee^{\leq} \mathbb{V}$. Uma imagem em níveis de cinza I é uma função $I : \mathbb{P} \to \mathbb{V}$ onde o domínio $\mathbb{P}$ (comumente chamado de *pixels*) possui relação de adjacência, e o contradomínio são os níveis de cinza da imagem.

Seja $v \in \mathbb{V}$. A função de limiar para um valor v é definida como

$$\left| \begin{array}{ll} \lambda_v : & \mathbb{V}^{\mathbb{P}} \longrightarrow 2^{\mathbb{P}} \\ & \mathcal{F} \longrightarrow \{x \in \mathbb{P} : v \leq \mathcal{F}(x)\} \end{array} \right. \tag{1}$$

O limiar para um nível de cinza v para uma imagem I é definido como

$$\Lambda_v = \lambda_v(I)$$

O conjunto de componentes conexas é

$$\Theta_v = \Pi(\Lambda_v)$$

Seja o intervalo $[\![\alpha, \beta]\!] \subseteq \mathbb{V}$, o conjunto de todas as componentes conexas para os valores de v é definido como

$$\Theta_{[\![\alpha,\beta]\!]} = \bigcup_{v \in [\![,\alpha,\beta]\!]} \Theta_v$$

Portanto, o conjunto de todas as componentes conexas para todas os limiares é denotado por

$$\Theta = \Theta_{[\![\perp,\top]\!]}$$

A partir da definição de Θ agora podemos introduzir o conceito de árvore de componentes. Considere a relação binária parcial $\subseteq$ em Θ, ou seja, considere o conjunto de ordem parcial $(\Theta, \subseteq)$. Note que tal conjunto é considerado de ordem hierárquica superior devido ao conceito apresentado em 3.5, poís:

1. O máximo de tal conjunto corresponde a $\mathbb{P} = \bigvee^{\subseteq} \Theta$
2. Para qualquer componente $X_v \in \Theta_v$ ($v \in [\![\perp + 1, \top]\!]$) existe exatamente uma componente $X_{v-1}\ in \Theta_{v-1}$ tal que $X_v \subseteq X_{v-1}$. Portanto $(\subseteq (X), \subseteq)$ é totalmente ordenado

Conforme apresentado em 3.5, o diagrama de Hasse de $(\Theta, \subseteq)$ é uma árvore. Como essa árvore construida armazena a informação das componentes conexas, a ela denominamos de árvore de componentes, para ser mais específico a árvore construida é a *max tree*.

Definição 5. A *max tree* $\mathscr{T}$ é o diagrama de Hasse $(\Theta, \lhd)$ do conjunto de ordem parcial $(\Theta, \subseteq)$. Os elementos de Θ são os nós da árvore. A raiz é $\bigvee^{\subseteq} \Theta$. As folhas são os elementos de $\Delta^{\subseteq} \Theta$. As arestas são os elementos de $\lhd$.

A título de curiosidade a *max tree* possui uma árvore dual chamada de *min tree*, em que para sua construção basta substituir o operador $\leq$ por $\geq$ na equação (1)

É fácil notar que essa definição de *max tree* não é computacionalmente eficiente em termos de memória. Em [4] uma versão computacionalmente eficiente é adotada. Como o propósito desse projeto de pesquisa é apenas contextualizar o problema, não iremos abordar tal assunto.

3.7 Merge

Conforme explicado em 2, a árvore de componentes é amplamente utilizada para diversas operações. No entanto, quando a imagem é alterada se faz necessário reconstruir a árvore inteira. Nesse sentido [4] resolve parcialmente esse problema ao propor a operação de mescla (*merge*) entre os nós da árvore. Nessa seção iremos formular o problema da mescla com o intuito de fornecer o contexto para o objetivo dessa pesquisa.

Pergunta 1: sejam $I^- : \mathbb{P} \to \mathbb{V}$ uma imagem antes de sofrer alguma transformação, $\mathscr{T}^-_{\text{max}}$ a *max tree* associada a I^-, $C \subset \mathbb{P}$ um conjunto conexo, $a = \bigwedge^{\leq} I^-(C)$ e $b \in \mathbb{V}$ tal que $b > \bigvee^{\leq} I^-(C)$. A imagem I^+ é definida como:

$$\left| \begin{array}{lll} I^+ : & \mathbb{P} & \to \quad \mathbb{V} \\ & x \in \mathbb{P} \setminus C & \to \quad I^-(x) \\ & x \in C & \to \quad b \end{array} \right.$$

Seja $\mathscr{T}^+_{\text{max}}$ a *max tree* associada a I^+. Sob essas condições, como devemos modificar $\mathscr{T}^-_{\text{max}}$ para obter $\mathscr{T}^+_{\text{max}}$?

Pergunta 2: sejam $\mathscr{T}^-_{\text{min}}$ a *min tree* de I^- e $\overset{\triangle}{\mathscr{T}}{}^-_{\text{min}}$ uma subárvore da min tree. Seja $\mathscr{T}^+_{\text{min}}$ a min tree obtida ao se remover a súbárvore $\overset{\triangle}{\mathscr{T}}{}^-_{\text{min}}$ de $\mathscr{T}^-_{\text{min}}$. Seja I^+ a imagem reconstruida a partir de $\mathscr{T}^+_{\text{min}}$. Como modificar $\mathscr{T}^-_{\text{max}}$ para obter $\mathscr{T}^+_{\text{max}}$ sob essas condições?

4 Objetivos

Queremos resolver o problema dual que foi proposta na **Pergunta 1**, isto é, sejam $I^- : \mathbb{P} \to \mathbb{V}$ uma imagem antes de sofrer alguma transformação, $\mathscr{T}^-_{\mathsf{max}}$ a *max tree* associada a I^-, $C \subset \mathbb{P}$ um conjunto conexo, $a = \bigwedge^{\leq} I^-(C)$ e $b \in \mathbb{V}$ tal que $b \leq \bigvee^{\leq} I^-(C)$. A imagem I^+ é definida como:

$$\left| \begin{array}{lll} I^+ : & \mathbb{P} & \to \quad \mathbb{V} \\ & x \in \mathbb{P} \setminus C & \to \quad I^-(x) \\ & x \in C & \to \quad b \end{array} \right.$$

Pergunta de Pesquisa: Seja $\mathscr{T}^+_{\mathsf{max}}$ a *max tree* associada a I^+. Sob essas condições, como devemos modificar $\mathscr{T}^-_{\mathsf{max}}$ para obter $\mathscr{T}^+_{\mathsf{max}}$?

Resolver esse problema dual significa elaborar um algoritmo eficiente de divisão de nós em uma *max tree*. No entanto, para resolver esse problema faz-se necessário responder perguntas de pesquisa auxiliares sobre modelagem teórica e topológica, complexidade algorítmica e estrutura de dados, manutenção dinâmica de atributos, generalização e validação.

Pergunta sobre modelagem teórica: Como definir formalmente a reestruturação das componentes conexas e a redistribuição de arestas quando a redução de valores de cinza em um conjunto de píxeis provoca a fragmentação de um nó da max-tree em múltiplos nós-filhos?

Pergunta sobre complexidade algorítmica e estruturas de dados: Quais estruturas de dados e rotinas de rastreamento local permitem limitar a complexidade da operação de split, evitando que ela degrade para o custo de uma reconstrução $O(n \log n)$?

Pergunta sobre manutenção dinâmica de atributos: De que maneira os atributos dos nós (como área, momentos e caixas delimitadoras) podem ser recalculados de forma local e incremental quando a massa de píxeis de um nó é particionada entre novas ramificações da árvore?

Pergunta generalização: Como combinar as operações de merge (já resolvidas) e de split em um modelo unificado que suporte alterações arbitrárias e simultâneas de níveis de cinza em pontos isolados da imagem? Existem outros operadores necessários para generalizar?

Pergunta validação: Qual é o ganho de desempenho computacional obtido com o operador de split em filtros conectados complexos, granulometrias e no processamento interativo de imagens de grande escala em relação aos métodos tradicionais de reconstrução

5 Metodologia

Como mencionado na seção anterior, não basta apenas elaborar o algoritmo que reponsde a pergunta de pesquisa. É necessário garantir que o algoritmo proposto que responde a pergunta de pesquisa funcione em todas as instâncias do problema e possua performance melhor do que reconstruir a árvore de componentes. Posto isso, nesta seção iremos propor os passos necessários para garantir que o algoritmo desenvolvido seja eficiente e funcione para todas as intâncias do problema

Primeiramente, iremos pesquisar e construir um arcabouço teórico. Esse arcabouço fornece o rigor matemático necessário para garantir que o algoritmo que será desenvolvido possua a corretude e eficiência desejada. Não somente é necessário construir tal arcabouço, como também é necessário garantir que tal arcabouço esteja correto. Portanto uma etapa de demonstração de tal arcabouço se faz necessária.

Segundamente, em posse de tal alicerce matemático, será possível construir o algoritmo que realiza a fragmentação de nós em uma *max tree*. Caso não seja possível elaborar tal algoritmo de forma eficiente, esse alicerce funcionará também para explicar e demonstrar o porquê não é possível. Caso seja possível a elaboração do algoritmo, também é necessário demonstrar a sua corretude e a sua complexidade computacional em termos de espaço e de tempo. Nessa etapa um subetende-se que um pseudo-código será gerado.

Terceiramente, esse algoritmo será implementado em alguma linguam de computador para demonstrar a viabilidade física de implementação. Não somente isso, utilizaremos esse algoritmo implementado em datasets para coletar métricas que irão servir de comparação entre o

algoritmo desenvolvido e a reconstrução de *max tree*. Claramente antes de realizar tais testes um estudo de quais métricas serão selecionadas deve ser realizado previamente.

Finalmente, utilizaremos um estudo de caso com o algoritmo proposto para demonstrar praticamente em quais contextos ele pode ser utilizado.

6 Plano de trabalho e cronograma

7 Mitigação de Riscos

Referências

[1] Wonder A.L. Alves, Charles F. Gobber, Dennis J. Silva, Alexandre Morimitsu, Ronaldo F. Hashimoto, and Beatriz Marcotegui. Image segmentation based on ultimate levelings: From attribute filters to machine learning strategies. *Pattern Recognition Letters*, 133:264–271, May 2020.

[2] D. L. Farfan Cabrera, N. Gogin, D. Morland, B. Naegel, D. Papathanassiou, and N. Passat. Segmentation of Axillary and Supraclavicular Tumoral Lymph Nodes in PET/CT: A Hybrid CNN/Component-Tree Approach. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 6672–6679, Milan, Italy, January 2021. IEEE.

[3] Gregory M. Hunter and Kenneth Steiglitz. Operations on Images Using Quad Trees. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):145–153, April 1979.

[4] Wonder Alexandre Luz Alves, Nicolas Passat, Dennis J. Silva, Alexandre Morimitsu, and Ronaldo F. Hashimoto. Component Tree: Update Rather than Rebuild. *Journal of Mathematical Imaging and Vision*, 68(5):61, August 2026.

[5] Cyril Meyer, Étienne Baudrier, Patrick Schultz, and Benoît Naegel. Combining Max-Tree and CNN for Segmentation of Cellular FIB-SEM Images. In Bertrand Kerautret, Miguel Colom, Adrien Krähenbühl, Daniel Lopresti, Pascal Monasse, and Benjamin Perret, editors, *Reproducible Research in Pattern Recognition*, volume 14068, pages 77–90. Springer Nature Switzerland, Cham, 2023.

[6] L. Najman and M. Couprie. Building the Component Tree in Quasi-Linear Time. *IEEE Transactions on Image Processing*, 15(11):3531–3539, November 2006.

[7] L. Najman and M. Schmitt. Geodesic saliency of watershed contours and hierarchical segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(12):1163–1173, December 1996.

[8] Laurent Najman and Jean Cousty. A graph-based mathematical morphology reader. *Pattern Recognition Letters*, 47:3–17, October 2014.

[9] Benjamin Perret. Inf-structuring Functions: A Unifying Theory of Connections and Connected Operators. *Journal of Mathematical Imaging and Vision*, 51(1):171–194, January 2015.

[10] Azriel Rosenfeld. Connectivity in Digital Pictures. *Journal of the ACM*, 17(1):146–160, January 1970.

[11] P. Salembier and L. Garrido. Binary partition tree as an efficient representation for image processing, segmentation, and information retrieval. *IEEE Transactions on Image Processing*, 9(4):561–576, April 2000.

[12] P. Salembier, A. Oliveras, and L. Garrido. Antiextensive connected operators for image and sequence processing. *IEEE Transactions on Image Processing*, 7(4):555–570, April 1998.

[13] P. Salembier and J. Serra. Flat zones filtering, connected operators, and filters by reconstruction. *IEEE Transactions on Image Processing*, 4(8):1153–1160, August 1995.

[14] Philippe Salembier and Michael Wilkinson. Connected operators. *IEEE Signal Processing Magazine*, 26(6):136–157, November 2009.

[15] S. Tanimoto and T. Pavlidis. A hierarchical data structure for picture processing. *Computer Graphics and Image Processing*, 4(2):104–119, June 1975.